

斐波那契数列各项间模运算速算方法

——基于计算机算法优化的实验探究

石宇澍

“斐波那契数列”由意大利数学家莱昂纳多·斐波那契在1202年提出，起初用以解决兔子的繁殖问题，后因其简洁的递推结构、丰富的数论性质，成为数论、组合数学、代数等领域的经典研究对象。

斐波那契数列通常记为 $\{F_n\}$ ，初始条件 $F_0 = 0, F_1 = 1$ ，递推关系 $F_n = F_{n-1} + F_{n-2}$ 。斐波那契数列取模运算的研究最早可追溯至18世纪，1774年，拉格朗日（Joseph Louis Lagrange）首次注意到斐波那契数列模10的剩余序列具有周期性。如今，斐波那契数列取模运算的规律被广泛应用于密码学、素性测试等领域。

现有研究大多关注 $F_a \bmod x$ ，（其中 x 为任意整数）的计算问题，本文则另辟新径，研究斐波那契数列各项间模运算，即 $F_a \bmod F_b$ ($a, b \in \mathbb{N}$) 的速算方法。

一、现有计算方法

面对计算 $F_a \bmod F_b$ 任务，根据现有办法，可先用快速翻倍法（或矩阵快速幂法）计算出 F_b 的值，然后再用快速翻倍法计算 F_a ，并在每次计算中取模 F_b 。理论时间复杂度为 $O(\log n)$ 。也可以根据斐波那契数列加法公式 $F_{m+n} = F_{m+1}F_n + F_mF_{n+1}$ 和整除性质 $a|b \Rightarrow F_a|F_b$ ，设 $a = qb + r$ ，则 $F_a \equiv F_{(b-1)^q F_r} \pmod{F_b}$ ，将数据规模进一步缩小。

然而，考虑到斐波那契数列的增长速度，在实际计算时， $O(\log n)$ 的理论时间复杂度不能正确反映计算效率。对于64位计算机，无符号整数上限为 $2^{64} \approx 1.8 \times 10^{19}$ ，如果对一个大于这个上限的数字进行运算，无法从算术逻辑单元（ALU）硬件层面完成，必须在软件上进行大数操作。在C中，没有原生的大数操作的函数，必须手动编写模拟竖式进行运算，加减法需要遍历较长数的每一位，乘除法（取模）需要分别嵌套遍历两个数字的每一位。设数据位数为 k 、对每一位数字的计算为 $O(1)$ ，则对大数进行加法（减法）时，时间复杂度为 $O(k)$ ，对大数进行乘除法（取模）时，时间复杂度为 $O(k^2)$ 。斐波那契数列增长速度很快，自 $n \geq 94$ 起， F_n 就超过64位无符号整数上限。根据通项公式 $F_n = \frac{1}{\sqrt{5}} \left[\left(\frac{1+\sqrt{5}}{2} \right)^n - \left(\frac{1-\sqrt{5}}{2} \right)^n \right]$ ，对其取10的对数， $\log_{10} F_n = n \log_{10} \varphi - \frac{1}{2} \log_{10} 5$ ，（其中 $\log_{10} \varphi$ 是非负的常数），可见斐波那契数列 $\{F_n\}$ 的位数与 n 成正比，故而，考虑到软件层面的大数运算，如果使用传统方法计算 $F_a \bmod F_b$ ，计算效率

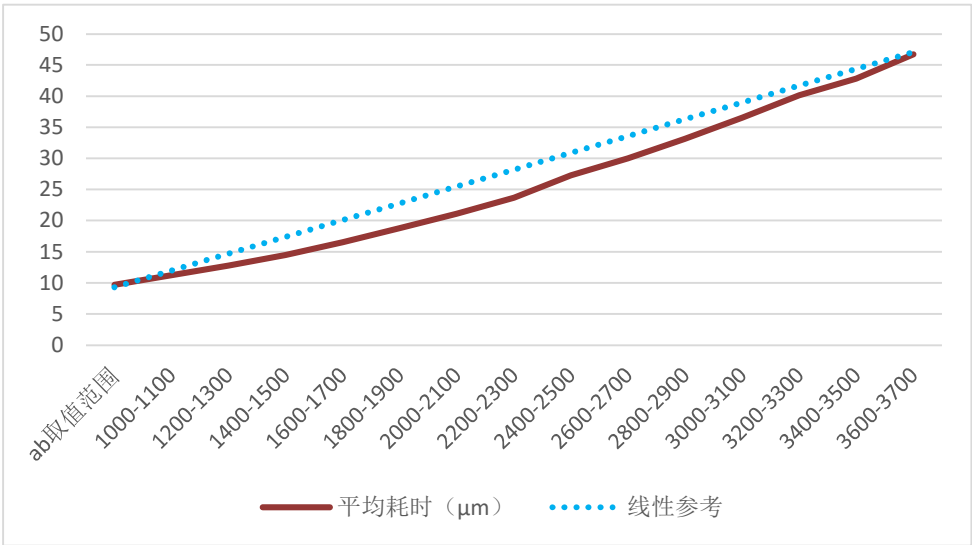
必然受到大数操作的影响。具体来说，设大数操作层面每一次操作为 $O(1)$ ，且快速翻倍法中每次迭代（或递归）也为 $O(1)$ ，则当我们先用快速翻倍法（或矩阵快速幂法）计算出 F_b 的值，然后再用快速翻倍法计算 F_a ，并在每次计算中取模 F_b 时，最终时间复杂度为 迭代（或递归）次数 $O(\log n) \times$ （每次取模复杂度 $O(n^2)$ + 每次按公式大数乘法计算复杂度 $O(n^2)$ ），即 $O(n^2 \log n)$ ；如果利用上文提到的同余式缩小数据规模再计算，则会牵扯进大数的幂运算，更加降低了效率，反而得不偿失。

不同于 C，Python 具有原生的大数操作的能力，且使用了很多优化算法。例如，Python 会将大数分割为若干个 digit 块分别计算，且针对数据大小自动使用 Karatsuba、FFT 等算法，最理想情况下可以将大数乘除的时间复杂度压缩至 $O(n \log n)$ 。上表所示是笔者使用 Python，对指定范围的 a, b 使用传统方法计算 $F_a \bmod F_b$ ，借助 time 函数统计每次运算平均消耗的时间（时间单位为 μm ，每次计算重复 30 次取平均值，耗时是指对于每个 a, b 计算 $F_a \bmod F_b$ 消耗的时间）

从图中可以看出，其增长快于线性。我们假设大数乘除的时间复杂度为理论下限 $O(n \log n)$ ，考虑快速翻倍法求每个 F_a, F_b 的时间消耗（ $O(\log n)$ ），则传统方法计算 $F_a \bmod F_b$ 的时间复杂度为 $O(n \log n \log n)$ 。

#	1	2	3	4	5	6	7
ab 取值范围	1000-1100	1200-1300	1400-1500	1600-1700	1800-1900	2000-2100	2200-2300
平均耗时(μm)	9.6981	11.2142	12.7895	14.4799	16.5298	18.7729	21.0815
8	9	10	11	12	13	14	15
2400-2500	2600-2700	2800-2900	3000-3100	3200-3300	3400-3500	3600-3700	3800-3900
23.6509	27.2517	29.9907	33.1383	36.5346	40.1496	42.8028	46.7385

绘图如下：



二、本文介绍的速算方法

是否有更好的方法计算 $F_a \bmod F_b$? 在实践过程中, 笔者发现、总结、并证明了下面这个高效的闭式算法。

$$F_a \bmod F_b = 0, \text{ 当 } (a = 1 \wedge b = 2) \vee (a = 3 \wedge b = 2) \vee (a = 5 \wedge b = 2) \vee (a = 7 \wedge b = 2); F_a \bmod F_b = F_a, \text{ 当 } a < b; \text{ 否则}$$

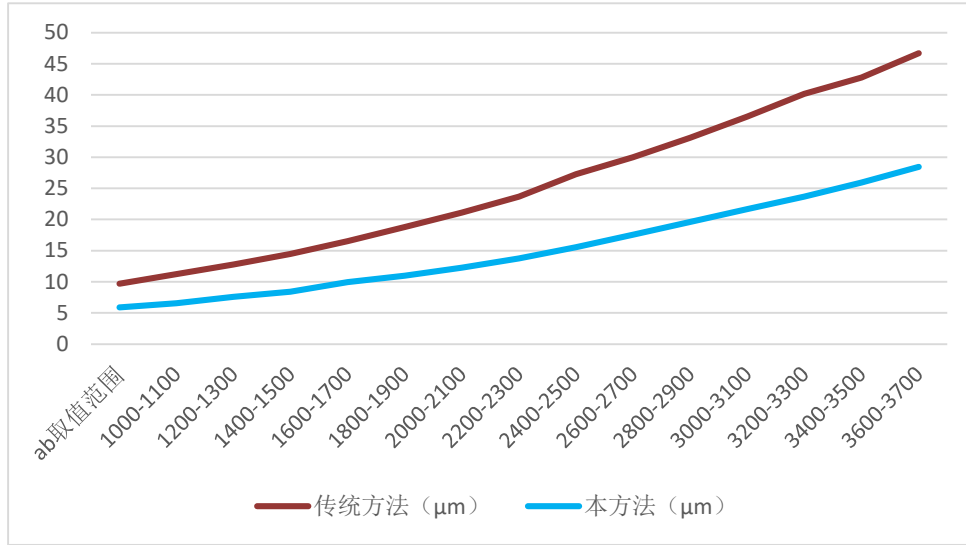
$$F_a \bmod F_b = \begin{cases} F_a \bmod 4b, & b \text{ 奇} \\ F_a \bmod 2b, & b \text{ 偶} \\ F_b - F_{2b-a}, & b \text{ 奇} \wedge a \text{ 奇} \wedge b \leq a \leq 2b \\ F_{2b-a}, & ab \text{ 异奇偶} \wedge b \leq a \leq 2b \\ F_b - F_{a-2b}, & b \text{ 奇} \wedge 2b < a \leq 3b \\ F_{4b-a}, & b \text{ 奇} \wedge a \text{ 奇} \wedge 3b < a < 4b \\ F_b - F_{4b-a}, & b \text{ 奇} \wedge a \text{ 偶} \wedge 3b < a < 4b \\ F_b - F_{2b-a}, & b \text{ 偶} \wedge a \text{ 偶} \wedge b \leq a < 2b \end{cases}$$

根据此, 我们可以把两个斐波那契数列中的数字的取模运算 ($F_a \bmod F_b$) 转化为加减法运算。对 F_n 进行加减法运算的时间复杂度理论下限为 $O(n)$ (设对每一位数字的计算为 $O(1)$), 考虑快速翻倍法的时间消耗 ($O(\log n)$), 则本方法计算 $F_a \bmod F_b$ 的时间复杂度为 $O(n \log n)$. 当然, 如果考虑快速翻倍法中的大数乘法, 这个表述可能不太准确。

作者同样使用 Python, 对指定范围的 a, b 使用本方法计算 $F_a \bmod F_b$, 借助 time 函数统计每次运算平均消耗的时间 (时间单位为 μm , 每次计算重复 30 次取平均值, 耗时是指对于每个 a, b 计算 $F_a \bmod F_b$ 消耗的时间) 从图中可以看出, 其效率优于传统方法。面对更大的数据规模, 优势可能更加明显。在特定的使用场景中, 我们有机会预计算出足够使用的斐波那契数列的数值存储在内存中, 不必每次计算都进行一次快速翻倍法, 这种情况下, 本方法的效率可能远远优于强行取模。

#	1	2	3	4	5	6	7
ab 取值范围	1000-1100	1200-1300	1400-1500	1600-1700	1800-1900	2000-2100	2200-2300
平均耗时(μm)	5.9166	6.5495	7.5577	8.4091	9.9265	10.9893	12.2825
8	9	10	11	12	13	14	15
2400-2500	2600-2700	2800-2900	3000-3100	3200-3300	3400-3500	3600-3700	3800-3900
13.7502	15.5342	17.5263	19.5834	21.6512	23.6790	25.9098	28.4808

绘图如下:



三、数学证明

下面所有同余式都是对 $\text{mod } F_b$ 而言。

加法定理: $F_{m+n} = F_{m+1}F_n + F_mF_{n-1} \Rightarrow F_{n+b} \equiv F_nF_{b\pm 1}$, ①;

Cassini 恒等式: $F_{n+1}F_{n-1} - F_n^2 = (-1)^n \Rightarrow F_{b\pm 1} \equiv (-1)^b$, ②;

减法定理: $F_{m-n} = (-1)^n(F_mF_{n+1} - F_{m+1}F_n) \Rightarrow F_{b-n} \equiv (-1)^{n+1}F_{b+1}F_n$, ③;

1. 证明 $F_a \text{ mod } F_b = \begin{cases} F_a \text{ mod } 4b, & b \text{ 奇} \\ F_a \text{ mod } 2b, & b \text{ 偶} \end{cases}$

由①得, $F_{a+2b} = F_{(a+b)+b} \equiv F_{a+b}F_{b-1}$, 对于其中的 F_{a+b} , $F_{a+b} \equiv F_aF_{b-1}$,

所以 $F_{a+2b} \equiv F_{b-1}^2 F_a$, 由②得, $F_{a+2b} \equiv (-1)^b F_a$.

b 偶, $F_{a+2b} \equiv F_a$, 则命题显然可证得。

b 奇, $F_{a+2b} \equiv -F_a$, 再用一次上式, 得 $F_{a+4b} = F_{(a+2b)+2b} \equiv -F_{a+2b} \equiv -(-F_a)$, 即 $F_{a+4b} \equiv F_a$, 则命题显然可证得。

$$2. \text{ 证明 } F_a \bmod F_b = \begin{cases} F_b - F_{2b-a}, & b \text{ 奇 } a \text{ 奇 } \wedge b \leq a \leq 2b \\ F_b - F_{2b-a}, & b \text{ 偶 } a \text{ 偶 } \wedge b \leq a < 2b \\ F_{2b-a}, & ab \text{ 异奇偶 } \wedge b \leq a \leq 2b \end{cases}$$

设 $t = 2b - a$, 则 $a = 2b - t$. 由①得, $F_{2b-t} = F_{b+(b-t)} \equiv F_{b-t}F_{b+1}$, 由③得, 对于其中的 F_{b-t} , $F_{b-t} \equiv (-1)^{t+1}F_{b+1}F_t$, 所以 $F_{2b-t} \equiv (-1)^{t+1}F_{b+1}^2F_t$, 由②得, $F_{2b-t} \equiv (-1)^{t+1}(-1)^bF_t = (-1)^{3b-a+1}F_t$.

ab 同奇偶, $F_{2b-t} \equiv -F_t$, $F_a \equiv -F_{2b-a}$, 则命题显然可证得。

ab 异奇偶, $F_{2b-t} \equiv F_t$, $F_a \equiv F_{2b-a}$, 则命题显然可证得。

$$3. \text{ 证明 } F_a \bmod F_b = F_b - F_{a-2b}, \quad b \text{ 奇 } \wedge 2b < a \leq 3b$$

设 $t = a - 2b$, 则 $a = 2b + t$. 由①得, $F_{2b+t} = F_{b+(b+t)} \equiv F_{b+t}F_{b+1}$, 对于其中的 F_{b+t} , $F_{b+t} \equiv F_{b+1}F_t$, 所以 $F_{2b+t} \equiv F_{b+1}^2F_t$, 由②得, $F_{2b+t} \equiv (-1)^bF_t$, 即 $F_{2b+t} \equiv -F_t$, 即 $F_a \equiv -F_{a-2b}$, 则命题显然可证得。

$$4. \text{ 证明 } F_a \bmod F_b = \begin{cases} F_{4b-a}, & b \text{ 奇 } a \text{ 奇 } \wedge 3b < a < 4b \\ F_b - F_{4b-a}, & b \text{ 奇 } a \text{ 偶 } \wedge 3b < a < 4b \end{cases}$$

设 $t = 4b - a$, 则 $a = 4b - t$. 同 2. 之理, 可以得到 $F_{4b-t} \equiv (-1)^{4b-a+1}F_t$.

a 奇 b 奇, $F_{4b-t} \equiv F_t$, $F_a \equiv F_{4b-a}$, 则命题显然可证得。

a 偶 b 奇, $F_{4b-t} \equiv -F_t$, $F_a \equiv -F_{4b-a}$, 则命题显然可证得。

四、总结

本文发现了提高斐波那契数列中较大数互相取模 ($F_a \bmod F_b$) 运算效率的方法, 主要运用了数论知识。